
Oliver Krauss

Indexes and Schema Validation

MDB3 | MTD.ba VZ WS26

Hagenberg 13.10.2026

HAGENBERG | LINZ | STEYR | WELS



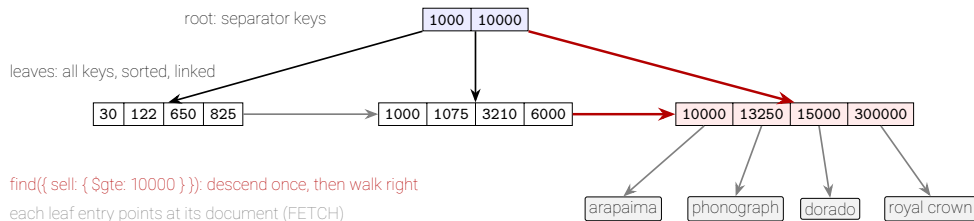
UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

explain: What the Server Did

```
db.items.find({ sell: { $gte: 10000 } }).explain("executionStats").executionStats
// { nReturned: 189, executionTimeMillis: 1,
//   totalKeysExamined: 0, totalDocsExamined: 4456,
//   executionStages: { stage: 'COLLSCAN', ... } }
```

- **COLLSCAN**: the server read **every** document to return 189. On 4,456 documents that is a millisecond; on 4 million it is the page timing out
- **totalDocsExamined** against **nReturned** is the number to watch. 4456 for 189 is the smell
- **explain** works on **find**, **aggregate**, **update**, **delete**. Ask it before you guess

How an Index Works: the B+ Tree



- The keys live **sorted** in the leaves; the inner nodes only say which way to go. A lookup descends once: three or four steps for millions of keys
- The leaves are **linked**, so a range or a sort is one descent and a walk along the chain, in order
- Every leaf entry points at its document. That is why **IXSCAN** is followed by **FETCH**, and why every insert has to place a key in every index

createIndex, and the Same Query Again

```
CREATE INDEX idx_sell ON items (sell);
```

```
db.items.createIndex({ sell: 1 }) // 'sell_1'
db.items.find({ sell: { $gte: 10000 } }).explain("executionStats").executionStats
// { nReturned: 189, totalKeysExamined: 189, totalDocsExamined: 189,
//   executionStages: { stage: 'FETCH', inputStage: { stage: 'IXSCAN', ... } } }
```

- The same B-tree as in *Relational Databases*: sorted keys with pointers to documents. **1** ascending, **-1** descending
- **IXSCAN** walks 189 keys, **FETCH** loads 189 documents. Examined equals returned: the ideal
- **_id** always has an index. Everything else is your decision, per collection, from the queries you actually run

Compound Indexes: Equality, Sort, Range

```
db.items.find({ type: "fish" }).sort({ sell: -1 }) // with sell_1 only: FETCH,  
            then SORT
```

```
db.items.createIndex({ type: 1, sell: -1 })  
db.items.find({ type: "fish" }).sort({ sell: -1 }).explain("executionStats")  
// nReturned: 80, totalKeysExamined: 80, totalDocsExamined: 80, FETCH > IXSCAN, no  
   SORT
```

- One index on several fields, in **this order**: walk **type** to **fish**, and the **sell** part is already sorted
- **ESR: E**quality first, then the **S**ort, then **R**anges. `{type: 1, sell: -1}` serves “fish by price” and “fish over 1000”
- Not “everything over 1000 by price”: without **type** the key has no start. A prefix works, a suffix does not

Arrays, Uniqueness, Text

```
db.villagers.createIndex({ furniture: 1 })           // one key per element
db.villagers.find({ furniture: 717 })    // IXSCAN, 10 keys, 10 documents

db.villagers.createIndex({ name: 1 }, { unique: true })
db.villagers.insertOne({ name: "Admiral", species: "Bird" })
// E11000 duplicate key error ... index: name_1 dup key: { name: "Admiral" }

db.items.createIndex({ name: "text", sourceNotes: "text" })
db.items.find({ $text: { $search: "guitar" } }, { name: 1, _id: 0 })
// rock guitar, electric guitar, acoustic guitar
```

- An array field gives a **multikey** index: every element is a key. The junction table's index, without the table
- **unique** is SQL's **UNIQUE**, enforced on insert and update. A text index is **MATCH ... AGAINST**: stemmed words, a score, one per collection. Where it ends: *Search I*

What an Index Costs

- **Every write pays.** An insert or update touches every index on the collection. Ten indexes: ten B-tree updates per insert
- **RAM.** Indexes work when they fit in memory; the working set is documents plus indexes. An index nobody uses still takes its share
- **Selectivity.** An index on **type** (25 values over 4,456 items) helps little; on **name** it finds one document. Index the fields your filters narrow with
- **Rule of thumb:** one index per query shape you run often, compound in ESR order, and **explain** to prove it. Then **dropIndex** what the plan never picks

Which Index?

With your neighbour. For each query, name the index that serves it, or say that none should be created and why.

1. `db.items.find({ type: "music" }).sort({ name: 1 })`
2. `db.villagers.find({ "birthday.month": 10 })`
3. `db.items.find({ sell: { $gte: 1000, $lte: 5000 }, type: "houseware" }).sort({ name: 1 })`
4. `db.villagers.find({ furniture: { $all: [717, 1849] } })`
5. `db.items.find({ diy: true })` where 1,300 of 4,456 items are diy

Which Index? Answers

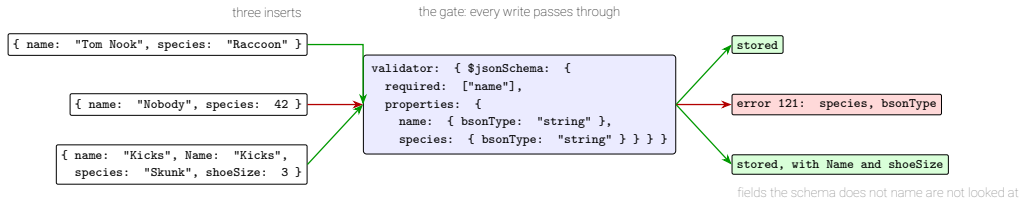
```
// 1: equality, then sort
{ type: 1, name: 1 }
// 2: a dot path is a field like any other
{ "birthday.month": 1 }
// 3: ESR: equality (type), sort (name), range (sell)
{ type: 1, name: 1, sell: 1 }
// 4: multikey; both values are looked up in the same index
{ furniture: 1 }
// 5: none. A third of the collection matches; the scan is cheaper than walking
//      1,300 index keys and fetching 1,300 documents one by one.
```

- Number 3 surprises: the range field goes **last**, even though it is in the filter, so the index can hand out the sort order

The Bill from Session One

- **Name** instead of **name**, "2400" instead of 2400, a **personality** of "Grumpy": accepted, and found weeks later by a query that returns too little
- In SQL the **INSERT** would have failed: **NOT NULL**, the column type, a **CHECK**, an **ENUM**
- MongoDB can refuse too. It is **opt-in**, per collection, and it is called a **validator**
- The trade: the flexibility that let **items** hold 25 shapes stays where you want it, and stops where you say

A Validator, as a Picture



- A validator is a **gate on the collection**: every insert and every update result is checked against it before it is stored
- It checks exactly the rules it names. **species: 42** breaks a rule; **Name** and **shoeSize** break none, because none mentions them
- Read that twice: the capital-N typo from session one walks straight through a validator that does not say **additionalProperties: false**

\$jsonSchema: Columns as a Document

```
db.runCommand({ collMod: "villagers", validator: { $jsonSchema: {
  required: ["name", "species", "personality", "birthday", "furniture"],
  properties: {
    name: { bsonType: "string", minLength: 1 },
    personality: { enum: ["Lazy", "Normal", "Cranky", "Snooty", "Jock", "Peppy",
                        "Smug", "Big Sister"] },
    birthday: { bsonType: "object", required: ["month", "day"], properties: {
      month: { bsonType: "int", minimum: 1, maximum: 12 } } },
    furniture: { bsonType: "array", items: { bsonType: "int" }, maxItems: 20 }
  } } }, validationLevel: "strict", validationAction: "error" })
```

- **required** is NOT NULL, **bsonType** the type, **enum** the ENUM, **minimum/maxItems/pattern** the **CHECK**. Unlisted fields stay free; **createCollection** takes the same option for a new collection

What a Refusal Looks Like

```
db.villagers.insertOne({ name: "Nobody", species: "Cat", personality: "Grumpy",
                        birthday: { month: 13, day: 1 }, furniture: ["717"] })
// MongoServerError: Document failed validation (code 121)
//   schemaRulesNotSatisfied: [ { operatorName: 'properties', propertiesNotSatisfied:
//     [
//       { propertyName: 'personality', details: [ { operatorName: 'enum',
//         reason: 'value was not found in enum', consideredValue: 'Grumpy' } ] },
//       { propertyName: 'birthday', ... 'maximum' ... consideredValue: 13 ... },
//       { propertyName: 'furniture', ... 'bsonType' ... consideredType: 'string' } ] }
//     ]
//   }
```

- Error 121, and **errInfo** names **every** rule the document broke, with the value it saw
- Updates are validated too: `$set: {personality: "Grumpy"}` on a valid document fails the same way
- In PHP it is a **BulkWriteException**; the write error carries the code and the same details

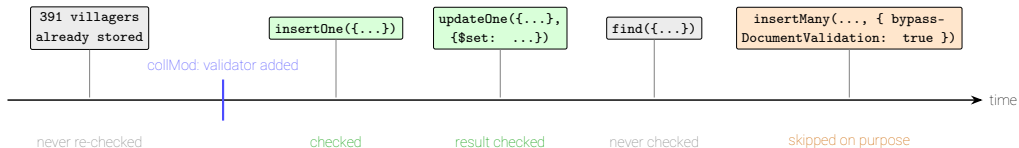
What a Schema Buys You

- **A contract.** The PHP app, the import script, the admin's mongosh session and next year's intern all write the same collection. The validator is the one place that says what a villager is, and it does not care who is writing
- **Errors at write time, not read time.** A refused insert is a stack trace today. A stored "2400" is a report that is 2,400 Bells short in December, found by nobody
- **Types the pipelines can trust.** `$avg` skips strings silently: an average over `sell` values that are half strings is an average of the other half. With `bsonType: "int"` that cannot happen
- **Documentation that is true.** Compass shows the validator on the collection; a wiki page shows what someone believed in March
- **Controlled evolution.** `schemaVersion` with an `enum` is how a migration knows it is finished, and how old code is stopped from writing the old shape

What a Schema Does Not Do

- **It ignores what it does not name.** Unlisted fields pass, misspelt fields pass.
`additionalProperties: false` closes that door, and with it the flexible schema for that collection
- **It knows one document.** No foreign key: `furniture: [717]` passes whether item 717 exists or not. No uniqueness: that is a unique index
- **It never looks back.** Documents stored before the validator, or imported with `bypassDocumentValidation`, stay as they are until something writes them
- **One schema per collection.** `items` holds 25 shapes; a fish needs `available`, a poster does not. That takes a `oneOf` per `type`, and it gets long
- **The point of documents was flexibility.** A validator that lists every field of every shape has rebuilt `CREATE TABLE` with more typing. Validate what must hold; leave the rest open

When the Validator Looks, as a Picture



- The validator runs at **write time**, on the document that would end up in the collection. Inserts, updates, replaces, upserts
- It does not run on reads, and it does not run on what was there before. A validator added today says nothing about yesterday's data
- **bypassDocumentValidation** is for imports of known-bad legacy data. Everywhere else it is a bug with a flag

The Documents That Are Already There

```
const schema = db.getCollectionInfos({ name: "villagers" })[0].options.validator.  
  $jsonSchema  
db.villagers.find({ $nor: [ { $jsonSchema: schema } ] }, { name: 1, _id: 0 })  
// Carrie, Cesar, Eunice, Hugh, Judy, Julian, Ozzie, Vic, Victoria    (more than 20  
  items)  
  
db.villagers.updateOne({ name: "Julian" }, { $set: { nickname: "Jules" } })  
// strict:    Document failed validation (maxItems: 20, numberOfItems: 26)  
db.runCommand({ collMod: "villagers", validationLevel: "moderate" })  
// moderate: matchedCount: 1, modifiedCount: 1
```

- A validator never checks what is already stored. `$jsonSchema` is a **query operator** too: find what would fail
- **strict** checks every write; **moderate** skips documents that were already invalid, so old data does not block unrelated updates
- **validationAction: "warn"** logs instead of refusing: the dry run

Which Is Rejected, and Why?

Your turn. The villager validator from the `$jsonSchema` slide, **strict** and **error**. Accepted or refused, and which rule?

1. `{ name: "Tom Nook", species: "Raccoon", personality: "Smug", birthday: { month: 5, day: 30 }, furniture: [717] }`
2. `{ name: "Isabelle", species: "Dog", personality: "Normal", birthday: { month: 12, day: 20 }, furniture: [], catchphrase: "!" }`
3. `{ name: "", species: "Cat", personality: "Lazy", birthday: { month: 1, day: 1 }, furniture: [1, 2] }`
4. `{ name: "Blathers", species: "Owl", personality: "Smug", birthday: { month: 4, day: 24.0 }, furniture: [1849] }`
5. `updateOne({name: "Admiral"}, { $push: { furniture: "787" } })`

Which Is Rejected? Answers

1. **Accepted.** Every required field, every type and value in range.
2. **Accepted.** An empty array is an array; `catchphrase` is not in `properties`, so it is free.
3. **Refused:** `name` has `minLength: 1`. A present but empty string is the classic gap `required` alone leaves open.
4. **Refused:** `24.0` is a double in the shell, and `day` asks for `int`. Type checks are exact; use `bsonType: "number"` if you do not care.
5. **Refused:** the update produces a document with a string in `furniture`; `items: {bsonType: "int"}` says no. Updates are validated on the result, not on the operator.

Changing the Shape Under a Validator

```
// v2: the birthday becomes one string "5-30", and every document carries its
    version
db.villagers.updateOne({ name: "Tom Nook" }, [
  { $set: { schemaVersion: 2, birthday: { $concat: [
    { $toString: "$birthday.month" }, "-", { $toString: "$birthday.day" } ] } } }
])
// Document failed validation: birthday, bsonType object, consideredType: 'string'
```

- An update can be a **pipeline**. **\$set** with expressions reads the old fields and computes the new ones, in place
- The validator says no, correctly: it still describes v1. In SQL this is the moment of **ALTER TABLE**
- Order matters: **first** widen the validator to accept both versions, **then** migrate, **then** narrow it to v2

A Validator for Two Versions, and the Migration

```
oneOf: [
  { not: { required: ["schemaVersion"] }, properties: { birthday: { bsonType: "
    object" } } },
  { required: ["schemaVersion"], properties: { schemaVersion: { enum: [2] },
    birthday: { bsonType: "string", pattern: "^[0-9]{1,2}-[0-9]{1,2}$" } } } ]
db.villagers.updateMany({ schemaVersion: { $exists: false } }, [ { $set: { ... } }
  ])
// matchedCount: 392, modifiedCount: 392    Admiral: birthday: '1-27', schemaVersion:
2
```

- **oneOf**: either v1 (no version field, birthday an object) or v2 (version 2, birthday a string)
- **updateMany** with a pipeline migrates in one statement; large collections in batches, or **lazily** per document touched. The version field makes that possible

Quick Retrieval: Test Yourself

Before the summary: can you answer these from memory?

1. Two numbers in **explain** tell you whether an index did its job. Which?
2. `{type: 1, sell: -1}` exists. Does `find({sell: {$gt: 1000}})` use it? Why?
3. What does an index on an array field contain?
4. Which SQL constraints does **\$jsonSchema** replace, with which keywords?
5. What does a validator do with documents that were in the collection before it was added?
6. In which order do you change the validator, migrate the data, and change the validator again?

Think first, then we go through the answers.

Quick Retrieval: Answers

1. **totalDocsExamined** against **nReturned**; and **COLLSCAN** versus **IXSCAN** in the stages.
2. No. **sell** is the second part of the key; without an equality on **type** the index has no starting point.
3. One key per array element (multikey), each pointing at the document.
4. **NOT NULL** = **required**, **type** = **bsonType**, **ENUM** = **enum**, **CHECK** = **minimum**, **maximum**, **pattern**, **maxItems**. No foreign key.
5. Nothing. They stay as they are; find them with **\$nor**: `[{$jsonSchema}]`, and **moderate** lets them be updated.
6. Widen (accept both versions), migrate (update with a pipeline, batch or lazy), narrow (v2 only).

Summary: Indexes and Schema Validation

Speed

- `explain("executionStats")`:
examined versus returned, **COLLSCAN**
versus **IXSCAN**
- **createIndex**: B-tree, one per query
shape, compound in ESR order
- Multikey for arrays, **unique** for
constraints, text for words
- Every index costs every write

Refusal

- **\$jsonSchema**: required, types, enums,
ranges, nested; error 121 names the rule
- Existing documents are untouched;
moderate and **warn** for the transition
- Evolution: version field, widen, migrate
with a pipeline update, narrow
- Next: what one document's atomicity
does and does not give you (*Transactions
and Consistency*)

Details: Reading a Plan

```
const e = db.items.find({ type: "fish" }).sort({ sell: -1 }).explain("executionStats")
// the plan chosen; rejectedPlans lists the
// others
e.queryPlanner.winningPlan
e.executionStats.executionStages // nested: FETCH > IXSCAN, or SORT > FETCH >
// IXSCAN
e.executionStats.executionStages.inputStage.indexName // 'type_1_sell_-1'
```

- Stages nest inside **inputStage**; read from the innermost out. **SORT** in the tree means an in-memory sort (100 MB limit)
- "**queryPlanner**" (default) shows the plan without running it; "**allPlansExecution**" runs every candidate
- Compass has the same as a tree under **Explain Plan**

Details: ESR Examples

Query	Index	Why
<code>{type: "fish"} sort sell</code>	<code>{type: 1, sell: -1}</code>	E then S
<code>{type: "fish", sell: {\$gt: 100}}</code>	<code>{type: 1, sell: 1}</code>	E then R
<code>{type: "fish", sell: {\$gt: 100}} sort name</code>	<code>{type: 1, name: 1, sell: 1}</code>	E, S, R
<code>{sell: {\$gt: 100}} sort type</code>	<code>{type: 1, sell: 1}</code>	S then R
<code>{type: {\$in: [...]}} sort sell</code>	<code>{type: 1, sell: -1}</code>	<code>\$in</code> counts as equality

- Sort direction must match the index direction or its exact mirror: `{a: 1, b: -1}` serves sort `{a: -1, b: 1}`, not `{a: 1, b: 1}`
- A query covered entirely by index fields (filter and projection) never fetches a document:
`totalDocsExamined: 0`

Details: Text Search

```
db.items.find({ $text: { $search: "cherry lamp" } },
              { name: 1, score: { $meta: "textScore" }, _id: 0 })
  .sort({ score: { $meta: "textScore" } }).limit(3)
// { name: 'cherry lamp', score: 1.5 } { name: 'cherry', score: 1 } { name: '
  Pisces lamp', score: 0.75 }
{ $text: { $search: "\"cherry lamp\"" } } // the phrase
{ $text: { $search: "cherry -lamp" } }    // cherry but not lamp
```

- Terms are ORed, stemmed per **default_language**, and scored by term frequency; the score is unbounded, not a percentage
- One text index per collection, up to many fields, with **weights**; **\$text** must be the top-level filter and cannot combine with a different sort
- No fuzziness, no phrase proximity, no facets: that is where Elasticsearch begins (*Search I*)

Details: Partial, Sparse, TTL, Wildcard

```
db.items.createIndex({ sell: 1 }, { partialFilterExpression: { type: "fish" } })
db.villagers.createIndex({ nickname: 1 }, { sparse: true })
db.visits.createIndex({ at: 1 }, { expireAfterSeconds: 60 * 60 * 24 * 30 })
db.items.createIndex({ "attributes.$**": 1 })
```

- **Partial**: only documents matching the expression are indexed; small, and used only by queries that include the filter
- **Sparse**: documents without the field are left out; with **unique**, this allows many documents without the field
- **TTL**: the server deletes documents once **at** is older than the limit; logs and sessions
- **Wildcard**: every field under a path, for documents whose attributes vary (the 25 shapes of **items**)

Details: One Validator for Many Shapes

```
validator: { $jsonSchema: {  
  required: ["name", "type", "sell"],  
  properties: { name: { bsonType: "string" }, sell: { bsonType: "int", minimum: 0 }  
    },  
  oneOf: [  
    { properties: { type: { enum: ["fish", "insect"] } }, required: ["available"] },  
    { properties: { type: { enum: ["houseware", "misc"] } }, required: ["buy"] },  
    { properties: { type: { enum: ["poster", "photo", "music"] } } }  
  ] } }
```

- The common part on top, then one **oneOf** branch per family of shapes, keyed by an **enum** on **type**
- **\$jsonSchema** is draft 4: no **if/then**. **oneOf** plus **enum** is the conditional
- Twenty-five shapes make a long validator; the alternative is one collection per family (*Document Modelling*)

Details: \$jsonSchema Keywords

Any type	<code>bsonType</code> (<code>string</code> <code>int</code> <code>long</code> <code>double</code> <code>number</code> <code>bool</code> <code>date</code> <code>objectId</code> <code>object</code> <code>array</code>), <code>enum</code>
Numbers	<code>minimum</code> , <code>maximum</code> , <code>exclusiveMinimum</code> , <code>multipleOf</code>
Strings	<code>minLength</code> , <code>maxLength</code> , <code>pattern</code> (a regular expression)
Objects	<code>required</code> , <code>properties</code> , <code>additionalProperties</code> : <code>false</code> , <code>minProperties</code>
Arrays	<code>items</code> , <code>minItems</code> , <code>maxItems</code> , <code>uniqueItems</code>
Logic	<code>allOf</code> , <code>anyOf</code> , <code>oneOf</code> , <code>not</code>

- It is JSON Schema draft 4 with BSON types added. **Not supported:** `format` (no `"email"`, no `"date-time"`), `$ref`, `default`, `definitions`. A date is `bsonType: "date"`; an e-mail is a `pattern`
- `additionalProperties: false` turns the flexible schema off for that collection: every field must be listed

Details: Validators with Query Operators

```
validator: { $and: [
  { $jsonSchema: { ... } },
  { $expr: { $lte: ["$sell", "$buy"] } },           // compare two fields
  { hemisphere: { $in: ["north", "south"] } }      // any query filter works
] }
```

- A validator is a **query filter** the document must match. **\$jsonSchema** is the usual one, but **\$expr**, **\$in**, **\$regex** and the rest are allowed
- **\$expr** is how you write a **CHECK** (**sell <= buy**) across fields, which JSON Schema alone cannot say
- Not allowed: **\$where**, **\$text**, **\$near**, and anything that reads other documents. There is still no foreign key

Details: Migration Strategies

- **Big bang:** one `updateMany` with a pipeline, in a maintenance window. Simple; locks nothing for long on MongoDB, but a million documents take minutes and the application must not write the old shape meanwhile
- **Batches:** the same update with a filter on `schemaVersion` and a `limit` loop; the application reads both versions during the run (`oneOf` in the validator)
- **Lazy:** the application upgrades a document when it reads it, writes it back as v2; the validator accepts both forever, a nightly batch finishes the stragglers. Zero downtime, two code paths
- **Never:** keep both shapes and let the code branch on `schemaVersion`. Common, and the reason the version field exists in the first place
- In every case: `bypassDocumentValidation: true` exists for imports of known-bad legacy data, and is a smell everywhere else